Design Phase: Building a Blog with Vite + React, Strapi, and Tailwind CSS

Tyler Sanderson

Wilmington University

Experiential Learning in CSC (22672.SEM)

Charles A. Bachman, PhD MBA PMP

March 1, 2025

Abstract

This document further discusses the technical architecture and technologies used to deploy our chosen tech stack, VRTS; we discuss some of the use-cases of the platform with high-level user-flow diagrams, the front-end design via wireframes; we discuss project requirements in relation to time, cost, technical feasibility, and flexibility/scalability of the platform; we then discuss some of the hardware/software best practices and ethical considerations that may come into play, such as GDPR, mainly addressing modular design, accessibility, and possible legal implications in our designs; finally, we provide a entity relationship diagram and relational schema to model all the currently required components of the platform.

Design Phase: Building a Blog with Vite + React, Strapi, and Tailwind CSS

Utilizing a modern tech stack - Vite+React, TailwindCSS, and Strapi, this project aims to develop a blog platform that can be easily maintained over time and serve as extra documentation to users' projects and portfolios as they learn and continue to grow; additionally, serving as a dynamic and interactive showcase of the skills and expertise I've learned throughout my computer science program. The future target audience for whom may be developers, students, or professionals looking for an interactive and visually appealing way to document their work and learning, but will be served locally with smaller ambitions of 2 home users for the time being, until later sprints are iterated to for a larger VPS deployment. The purpose of this document will be to define the technical architecture of our chosen tech stack, VRTS, as well as use-cases, front-end design, meeting requirements, best practices and ethical considerations, while also discussing a data dictionary of entity relationships and process logic to build an implementable plan for our development phase!

The client-side architecture consist of a front-end user interface built on top of Vite+React for its hot-module reloading and fast build times *(Why vite)*, and TailwindCSS for its efficient scalability and responsiveness *(TailwindCSS)*. By using a build tool on top of React, we still get all the benefits of component architecture. Each React component will represent different sections of the platform, allowing for clear naming schemes and API request moving forward, as well as ensuring future scalability and efficiency since we won't have to rebuild components each time we'd like to reuse them. We'll break components into sections like the navbar with links and search, the blogs landing page that will display some of the authors info and links, as well as recent or featured articles, sortable by tag and category. The authors blog page will contain components like the navbar, article card, article body with index, potential sharing and

comment system, and related posts. A static ‘about me’ page was originally planned to be implemented, but was found to be a bit redundant given the other applicable info on other pages, but will house the authors ‘about’ component and other applicable links more concisely. Pages will have to be called and loaded using Reacts Router, but will be managed by Strapi. To communicate with our back-end system, Strapi, which can utilize its own instance or a localized instance of SQLite, HTTP clients will use GraphQL API calls to fetch and display dynamic content such as blog posts and author info *(Strapi, Graphql API)*.

The server side will utilize Strapi for providing a web-based admin GUI for management of posts, users, and other data; as well as exposing API endpoints via GraphQL for the front-end to query. Strapi also provides built-in authentification for user roles; it supports and can manage database tools like SQLite or MongoDB to manage posts, users, tags, and categories. Strapi will also allow us to custom model content types like posts, potential comment section, and portfolio pages. This allows for future scalability, as defined in our requirements analysis.

Hosting for all the various tools in our stack can be hosted via local hardware, or a larger VPS system, as they are all fully compatible with various Linux implementations. This means the hosting can be implemented on local hardware via a local node inside a private lan, using Cloudflare for proxying HTTPS requests to the server, Nginx Proxy Manager for directing HTTPS traffic in and out of the local network, and a CloudflareDDNS attached to our Nginx instance to update our infrastructures IP address to Cloudflare, given our residential ISP service.

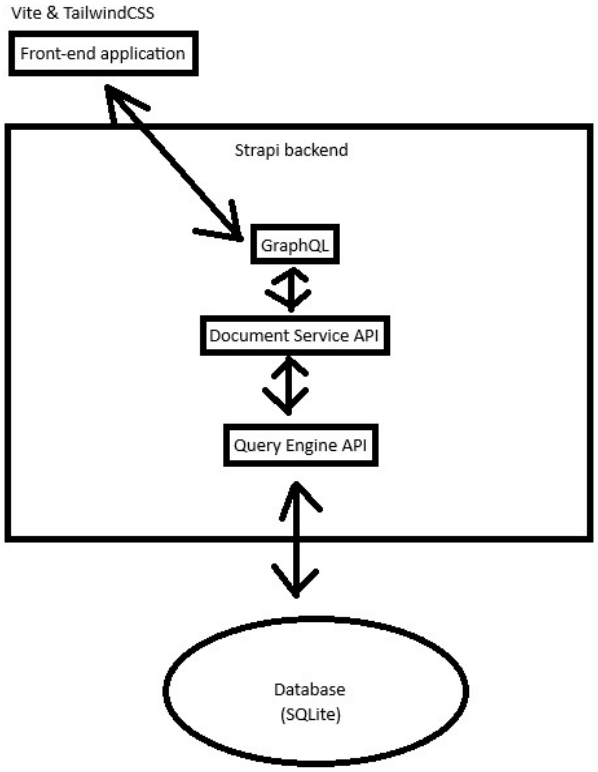
For version control, this project will be utilizing Git and GitHub, in tandem with Strapi’s content management solution, to ensure a safe and effective development environment. Continuous integration and deployment, or CI/CD, could more easily implemented in future iterations this way.

The data flow for this project has been greatly simplified in choosing Strapi for content management. The backend Strapi GUI allows content creators to add, update, delete posts, manage categories, tags, and various portfolio components, as well as content management plugins for embedding code and other nice-to-have features expected in a portfolio blog, a development blog nonetheless. As for real time updates, Strapi also contains plugins for an iteration focused on building out a comment and sharing system; but this will have to be reserved till after basic functionality is achieved with building our initial blog portfolio, as outlined in our requirements analysis.

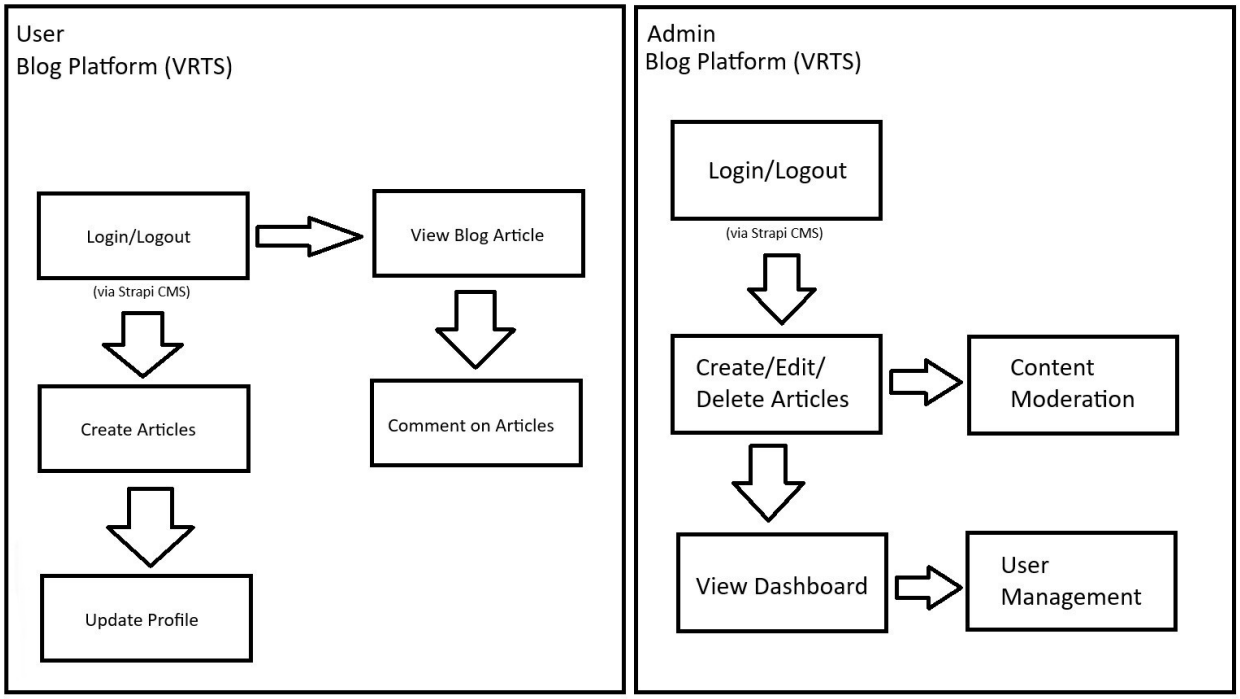
For our security, Strapi luckily handles user permissions and roles, as well as can be set to include OAuth for secure authorization. Future research will be needed after initial implementation to build a secure front-end portal for management and further security authorization, but the back-end GUI will be reserved to local management for the time being, so Strapi's built-in management will work for us for now. HTTPS will be handed out to clients via Nginx, specifically with LetsEncrypt, for ensuring safe transmission of data over SSL in and out of the network. This meets current demands, but scales well to fit global compliance policies. Using Cloudflare for proxying traffic to our endpoint also takes advantage of rate-limiting clients and API calls, as well as various other security controls that may scale well to future iterations, as GDPR compliance and similar compliance guidelines aren't necessarily a concern for the limited current user base of two users; but again, scales well to fit them in the future!

Essentially, the front-end and back-end are independent, but communicate via GraphQL APIs. Vite+React provide the dynamic and responsive user experience, while Strapi handles the content management and database, in our case SQLite, which is the default for Strapi. TailwindCSS ensures our components scale well on different devices and are user designed and

responsive.



(pwizla)



Currently, the users of this project will include administrators of the Strapi back-end and blog readers. Admins will use the Strapi web GUI to login/logout, moderate content, manage blog posts, manage users and accounts, as well as various dashboard analytics to track engagement! The blog viewer should be able to view posts and related links, but will be expanded in later iterations to include being to able to register//update/login/logout user accounts to deliver a managed comment system.

The main use-cases described are mainly handled by Strapi's CMS. Admins can create a new blog post by entering text, adding images, embedding code-snippets, selecting categories and tags, previewing the post, and submitting. Admins may also want to manage their authentication and authorization by visiting the Strapi back-end GUI to login, enter their credentials, the system authenticates and provides/denies access, the admin can logout at any time. The admin will also be able to manage content from the Strapi back-end where they may login to the admin dashboard, navigate to 'Posts Management' or 'Moderation', review posts,

approve, reject, edit, delete posts, and manage user activity, in future iterations. In these future iteration, users may want to login to their account, navigate to their profile page, update personal info like email or profile photo, and save changes in order to properly manage a compliant comment system.

Wireframing was conducted to address the functional requirements listed in our analysis phase. The functional requirements included:

- Shall display an ‘About-Me’ section, which will re-utilize other components of our blog when clicked.
- Shall display a a blog section with recent posts.
- Shall display social media buttons.
- Shall display links to portfolio and resume.
- Shall display Categories and Tags, in a drop-down.
- Must allow embedded code.
- Shall have a search function.
- Should feature a responsive design to accommodate a variety of devices.
- Must feature SEO Optimization.

Taking time, cost, and technical feasibility into account; Vite ensures we get a fast development environment since updates can be hot-swapped during development, React encourages re-usability and component building to reduce development time and improve accessibility in our design. These tools also ensure better maintainability since components and modules can be updated quickly. TailwindCSS allows us to use prebuilt elements directly in our JSX-React code. This reduces needed code and development time, while also encouraging accessibility and usability standards across multiple interfaces. Strapi provides an easy to use

admin dashboard, Content Management System, and customizable API to save time on developing a back-end for this project, given the scope and business implications described in the requirements analysis.

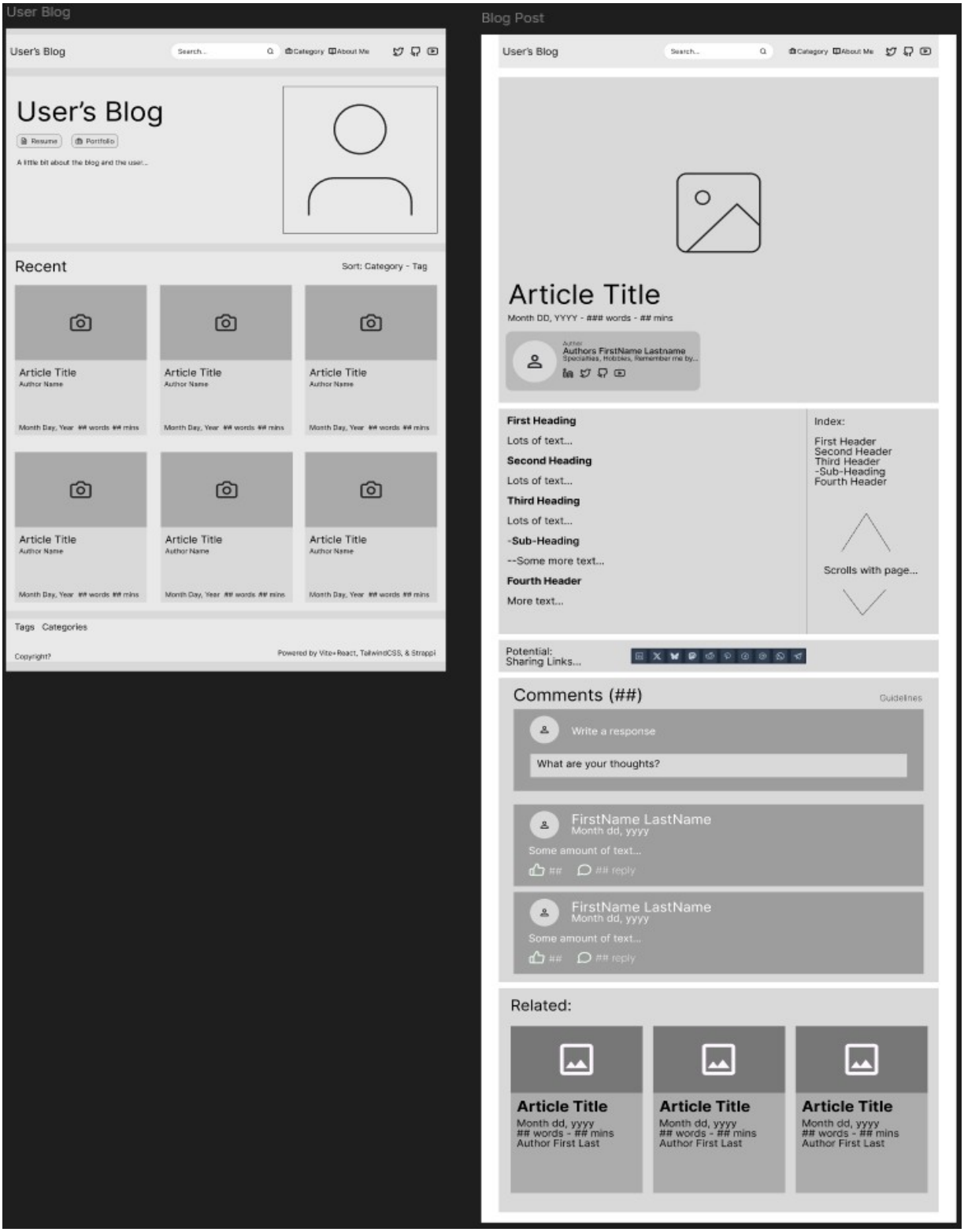
While there are licensing costs associated with using open-source software at scale and, especially, for profit, this projects current scope doesn't meet the criteria for self-hosting on my home-lab for the time being and does not currently intend to include any direct revenue streams, so our software stack is currently free to use *(Strapi, strapi / license)*! Furthermore, our stack is also widely and broadly supported, given the nature of open-source technologies. This means our project can still be more easily scaled to a larger VPS system, such as Hostinger or Linode, for quick, cheap, and scalable deployment should the projects scope and user base grow!

Technically, Vite is optimized for modern web standards, supports hot-module replacement, refered to as HMR, which speeds up development, debugging, and future iterations *(Why vite)*. React, and Strapi for that matter, offer libraries and plug-ins for further state-management, routing, and data retrieval; using React Router for navigation *(ReactRouter, Navigating)* and Axios for managing API request *(Axios API)*. TailwindCSS takes more of a 'mobile-first' philosophy, ensuring proper device scaling, as well as it still reduces query times and doesn't rely on custom CSS, but can still be overridden if necessary. Strapi's flexible API CMS ensures efficient management and maintanance, while also being easily integrated into our component based React app. Strapi also supports authentication, file uploads, user-roles, ensuring current security and functional standards, while allowing for future scalability!

In terms of flexiblity, by separating our front-end and back-end architectures via these GraphQL API calls, this also means that we can more easily experiment and swap out or upgrade components of our architecture without disrupting most of the functionality of the other systems!

This means if we'd like to experiment with different front-end build configurations, or we want to try another headless-CMS system, like Hugo, it should be much easier to implement should the needs arise *(Strapi, Graphql API)*.

Given the functional requirements, the non-functional requirements listed in our requirements analysis, and all that was discussed above, the following wireframe was designed to achieve a Minimum Viable Product that meets our current needs, while also playing with ideas for future iterations, such as the comment and social sharing functions.



There are still a few hardware and software best practices, as well as ethical

considerations to account for regarding our current and possible future scope. Some best practices include modular design, as satisfied by React and Strapi; code quality, such as using descriptive and consistent variable naming schemes and consistent organizational standards. Unit testing, which can be touched on more during our testing phase, can help test our API calls and security, such tools like Jest or React Testing Library. Using Git to keep track of our stack will also lend itself to testing and deployment.

In regards to accessibility on our blog portfolio, we can follow Web Content Accessibility Guidelines, or WCAG, during our development to encourage proper accessibility standards are being met to ensure a wide-audience has access to the platform *(WAI)*. TailwindCSS also utilizes ‘focus’ controls and screen reading tags, such as ‘sr-only’, to help meet accessibility standards *(Hover, Focus, & Other States)*. While more research will be performed before our testing phase, we can utilize tools like axe-core *(Deque, 2024)* or Lighthouse *(Introduction to lighthouse, 2016)* to help identify accessibility standards are being met. We can also try using various screen readers, like VoiceOver, or built-in tools to ensure accessibility.

TailwindCSS’s ‘mobile-first’ philosophy not only lends itself to accessibility standards and screen readers, but also ensures a mobile-friendly and responsive design. TailwindCSS’s text tagging controls also help adhere to accessibility, responsiveness, and SEO by sticking to standards and ensuring clearer controls.

Utilising Vite+React, we get lazy-loading and component based updates, reducing initial page loading times and improving SEO. We also ensure comparability with future optimization tools like WebP, with a tool like Image Optimization API, to reduce file sizes; GraphQL can efficiently call what we need already by its nature, so API calls are quick and support various

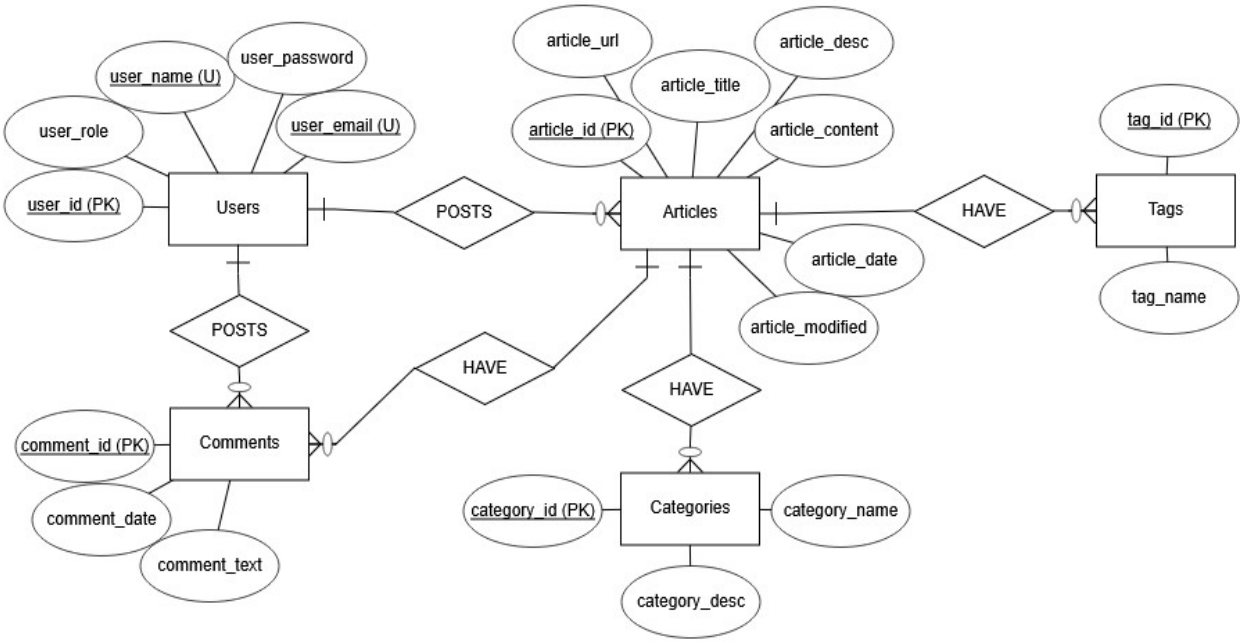
compression plugins. Strapi supports various content management delivery solutions for limiting file sizes and transfers.

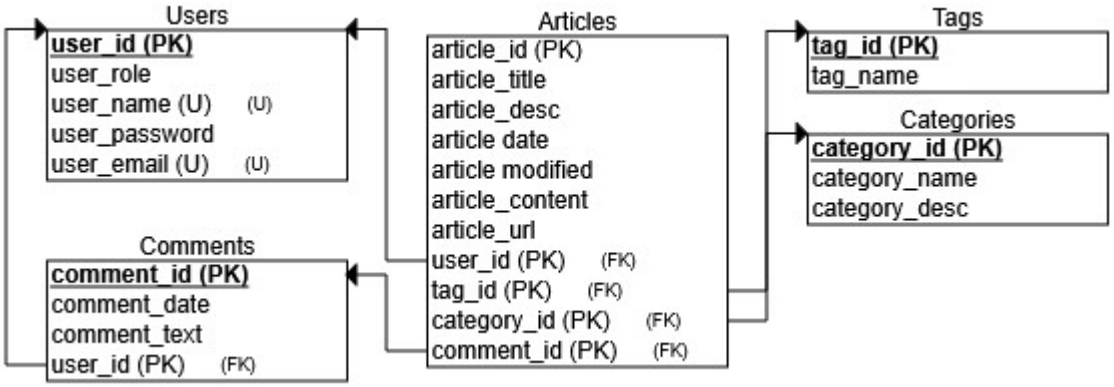
From a user data standpoint, our platform doesn't currently need to comply with GDPR regulations as the platform doesn't currently have data-collection or data-deletion concerns. By default, however, Strapi complies with said regulations by default in its content management system, so the platform should scale well to follow such standards *(Strapi.io)*. Encryption of admin authorization and like controls are handled by Strapi! Our communications will be handled and proxied over Cloudflare and funneled to our Nginx Proxy Manager with LetsEncrypt for HTTPS to aid in the current scopes admin privacy and encourage scalability and safe behavior and development. Further iterations will be needed for a more robust security audit given the current scope of the project.

With our use of open-source tools we get all the benefits of community, security, and scalability, but these tools do ask for attribution, and as discussed earlier in the document, may include licensing at larger scales not attained by this projects scope yet. However, with open-source software, by utilizing and leveraging these open-source tools for my project, I lead with the philosophy of community and documentation that seeks to give back to said community, which may lend itself to a larger role in its development, such as feature request and contributions in the future.

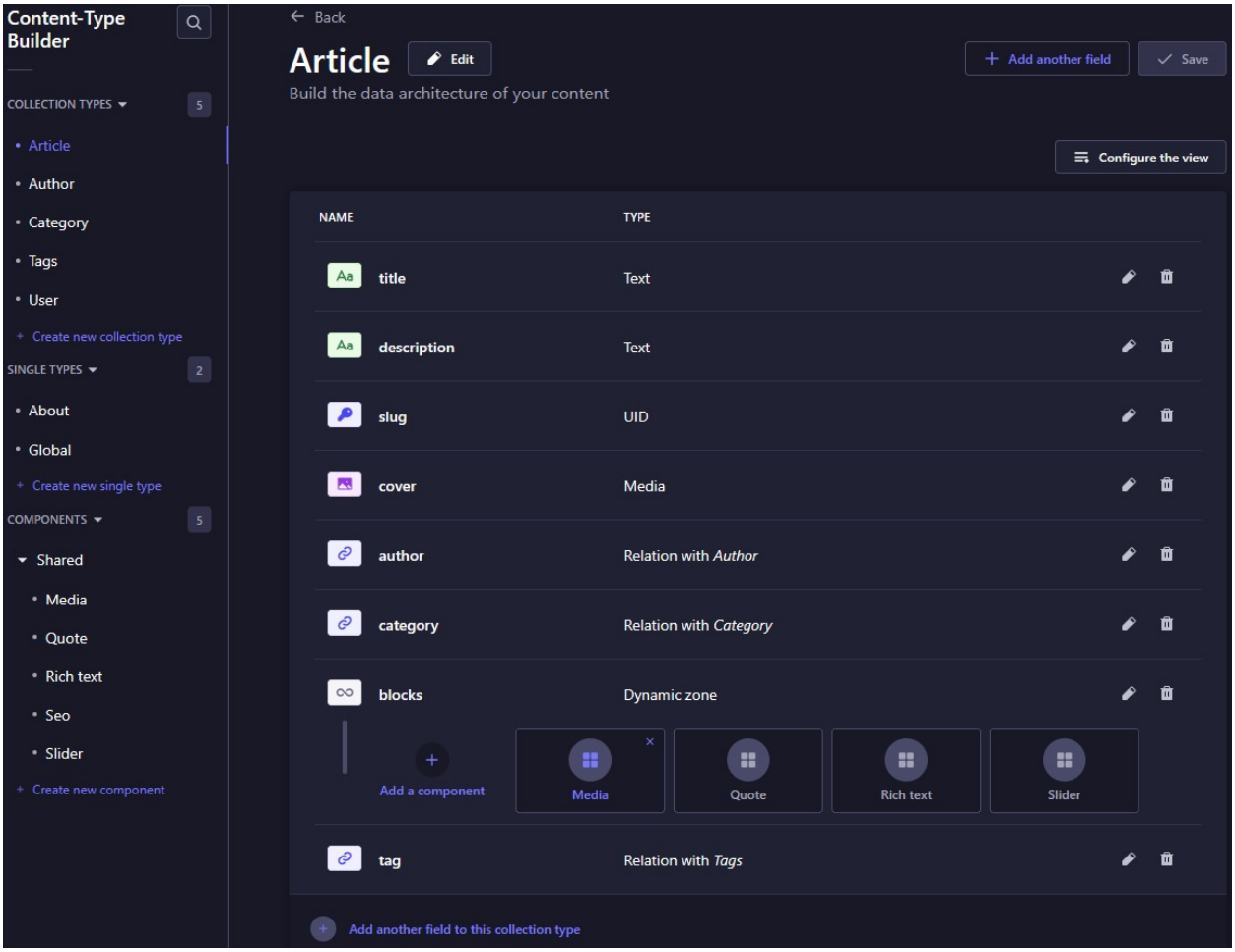
In conclusion, we defined the technical architecture of our chosen tech stack, VRTS, as well as use-cases by users, admins, and viewers; as well as the front-end design via wireframes, meeting requirements, best practices and ethical considerations, while also providing an Entity Relationship Diagram, process logic, and a data dictionary to build an implementable plan for our development phase!

Entity Relationship Diagram & Relational Schema of Strapi managed database - so-far!:





Installed Strapi CMS Dashboard:



References

(WAI), W. W. A. I. (n.d.). WCAG 2 at a glance. Web Accessibility Initiative (WAI). https://www.w3.org/WAI/standards-guidelines/wcag/glance/

Axe: Accessibility Testing Tools and software. Deque. (2024, June 3). https://www.deque.com/axe/

Axios API. Axios API | Axios Docs. (n.d.). https://axios-http.com/docs/api_intro

Hover, Focus, & Other States. Tailwind CSS. (n.d.-a). https://v2.tailwindcss.com/docs/hover-focus-and-other-states

Introduction to lighthouse. Chrome for Developers. (2016, September 27). https://developer.chrome.com/docs/lighthouse/overview

Navigating. React Router. (n.d.). https://reactrouter.com/start/library/navigating

pwizla. (n.d.). Content API: Strapi 5 documentation. Content API | Strapi 5 Documentation. https://docs.strapi.io/dev-docs/api/content-api

Rapidly build modern websites without ever leaving your HTML. Tailwind CSS. (n.d.-b). https://tailwindcss.com/

Stapi.io. (n.d.-a). Build modern websites with the most customizable Headless CMS. Strapi. https://strapi.io/

Strapi. (n.d.-b). Graphql API. Strapi 5 Documentation. https://docs.strapi.io/dev-docs/api/graphql

Strapi. (n.d.-c). strapi / license. GitHub. https://github.com/strapi/strapi/blob/main/LICENSE

Why vite. vitejs. (n.d.). https://vite.dev/guide/why.html